Retail × AI

für

Markant Gruppe

Tag 1

KI-Coding Masterclass

Vom Vibe Coding zum systematischen Context Engineering

Tag 1: Foundations & Cursor Workflows 10:00–16:00

Tag 2: Enterprise, Brownfield & CI/CD 09:00–15:00

Retail × AI

Agenda Tag 1: Foundations & Advanced Cursor Workflows

10:00–10:15	Ankommen & Check-In	
10:15–11:00	**Opener: Die 5 Levels der KI-Coding-Reife**	J-Kurve, Orchestrator-Mindset, Context-Fehler
11:00–11:45	**Greenfield-Challenge & BMAD-Workflow**	Kantinen-Planer Demo · Analyst→PM→Architect→Dev
11:45–13:00	Kaffeepause + Mittagessen	
13:00–14:00	**Spec-Driven Development (Level 3+)**	Requirements→Spec→Agent · Context Rot Warning
14:00–15:00	**Deep Dive: Cursor Rules (.mdc)**	markant-global.mdc · Live Exercise Teams
15:00–15:45	**Agent Modes, MCP & YOLO-Mode**	Plan/Agent/Ask/Debug/YOLO · terraform Allowlist
15:45–16:00	Wrap-up Tag 1	

10:00–16:00

Retail × AI

Agenda Tag 2: Enterprise Scale, Brownfield & Code Quality

09:00–09:15
Check-In & Recap Tag 1

09:15–10:30
Skalierung auf Enterprise-Codebases
Turbopuffer, AST-Chunking, .cursorignore

10:30–10:45
Kaffeepause

10:45–12:00
CI/CD-Integration & KI-gestütztes Testing
GitLab /mr · Background Agents · Bugbot

12:00–13:00
Mittagessen

13:00–14:15
Brownfield-Praxis: Legacy Code Refactoring
Plan→Test→Refactor auf echtem EDI-Code

14:15–14:30
Kaffeepause

14:30–15:00
Wrap-Up & Next Steps
5-Levels Re-Assessment · Cheat Sheet · Abschluss

09:00–15:00

Retail × AI

Tag 1 · 10:15

1

Die 5 Levels der KI-Coding-Reife

Opener · 10:15–11:00
Assistent > Orchestrator-Mindset

Retail × AI

The Five Levels: from Spicy Autocomplete to the Dark Factory

THE EVOLUTION OF AI-ASSISTED SOFTWARE DEVELOPMENT

SPICY AUTOCOMPLETE (Level 0)

The AI serves as a reference tool or enhanced search, like a smarter Stack Overflow. The developer manually writes all the code, using AI for specific snippets or answers.

CODING INTERN (Level 1)

The AI writes unimportant or boilerplate code. The developer prompts for specific functions but immediately reviews and integrates the output.

JUNIOR DEVELOPER (Level 2)

An interactive "pair programmer" partnership. The developer and AI trade off control, with the human reviewing code as it's generated in real-time.

DEVELOPER (Level 3)

The AI generates the majority of the codebase. The developer reviews everything the AI does, acting as the bottleneck for verification before progressing.

SENIOR DEVELOPER (Level 4)

The AI runs unattended for long periods, handling complex tasks. The human trusts the system's self-checks, only checking the final results much later.

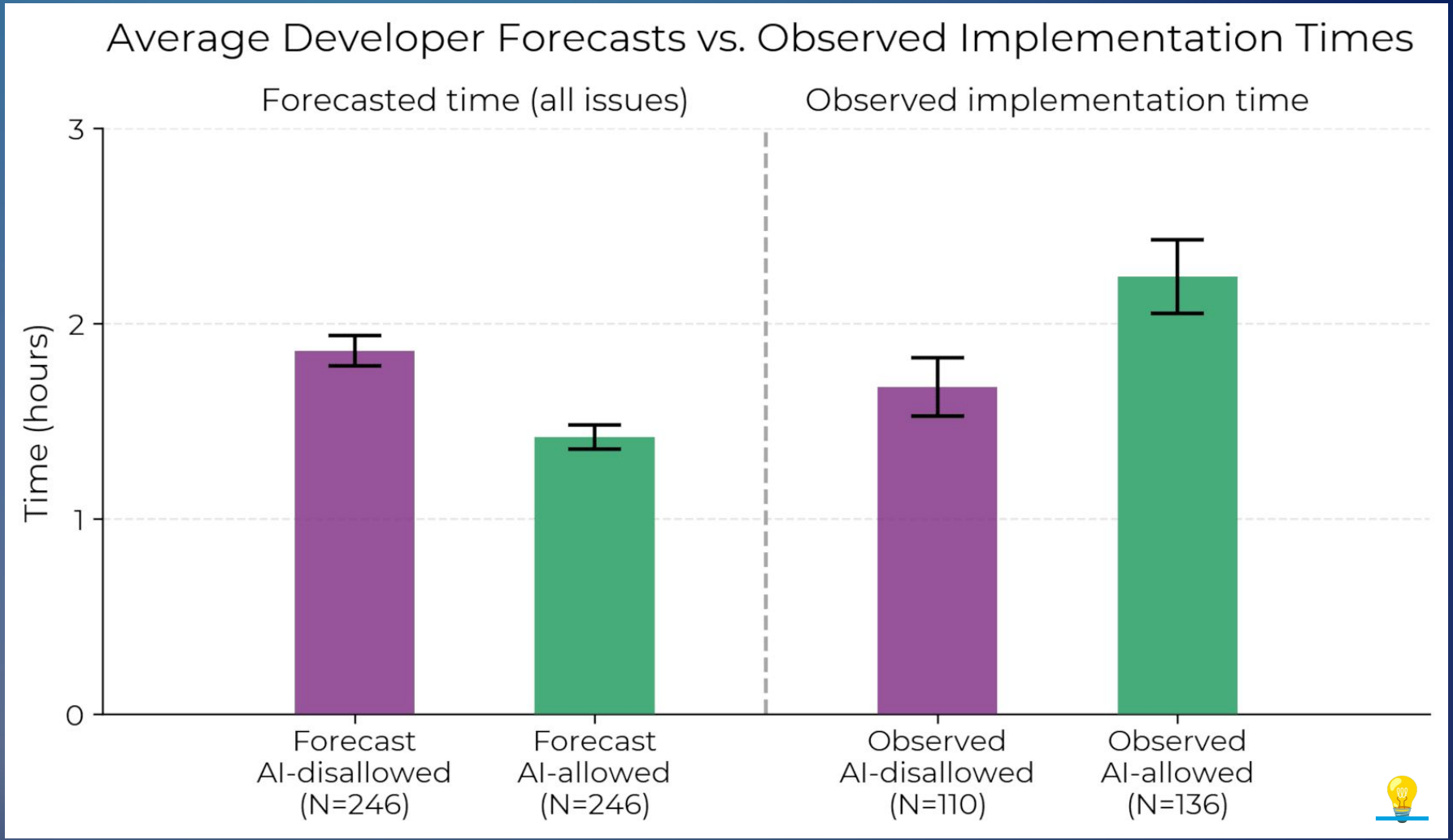
SOFTWARE FACTORY (Level 5)

The engineer manages the goals and the system, not the code. They provide plain English descriptions. The AI defines implementation, writes code, tests, fixes bugs, and ships.

Retail × AI

„METR: Erfahrene Entwickler fühlten sich mit KI +24% schneller, waren in dieser Studie aber tatsächlich 19% langsamer.“

Die Lücke zwischen Gefühl und Messung deutet darauf hin: Ohne neuen Workflow verschiebt KI Arbeit oft nur — vom Tippen zum Prüfen, Debuggen und Kontext-Management.“



Retail × AI

Die 5 Levels der KI-Coding-Reife

1	2	3	4	5
Spicy Autocomplete	**Tool User**	**Spec Engineer**	**Context Architect**	**Orchestrator**
Du schreibst Code, KI schlägt vor. Keine Überraschungen.	Tab-Completion, Chat für Snippets. Noch reaktiv.	Specs → Agent → Review. Du reviewst Ergebnisse, nicht Code.	.mdc Rules, Plans, MCP. KI kennt euren Stack.	Armee von Agents. Cloud + Parallel + CI/CD.
		Workshop	Workshop	Workshop

Dan Shapiro: Level 0–5 (Spicy Autocomplete → Dark Factory)
Workshop-Fokus: Level 1/2 → 3+ (Vibe Coding → systematisches Context Engineering)

Retail × AI

Tag 1 · 11:00

2

Greenfield-Challenge & BMAD-Workflow

Praxis · 11:00–11:45
"Kantinen-Planer" Demo

Retail × AI

Baue einen end2end MVP eines Kantinenplaner

10min

```
npx bmad-method install
```

ca. 10min

Retail × AI

Greenfield

BMAD-Workflow: Vom Chaos zur Spec in 25 Minuten

🍽 **Demo-Projekt: "Kantinen-Planer"**

1. Analyst	2. PM	3. Architect	4. Dev
Requirements sammeln	**PRD erstellen**	**API + Datenmodell**	**Story implementieren**
Menüs, Allergene, Bestellfenster, User-Roles procduct-brief.md	DEV User Stories, Akzeptanzkriterien, Prioritäten prd.md epics.md	REST-Endpoints, DB-Schema, Auth-Konzept **architecture.md**	Agent Mode + .mdc Rules → Code + Tests

Retail × AI

3

Spec-Driven Development

Level 3+ · 13:00–14:00
Context Rot Warning

Retail × AI

Warum Spec-Driven?

Der gleiche Task führt zu unterschiedlichen Interpretationen.

Ohne Spezifikation optimiert der Agent auf plausible Ausgabe.

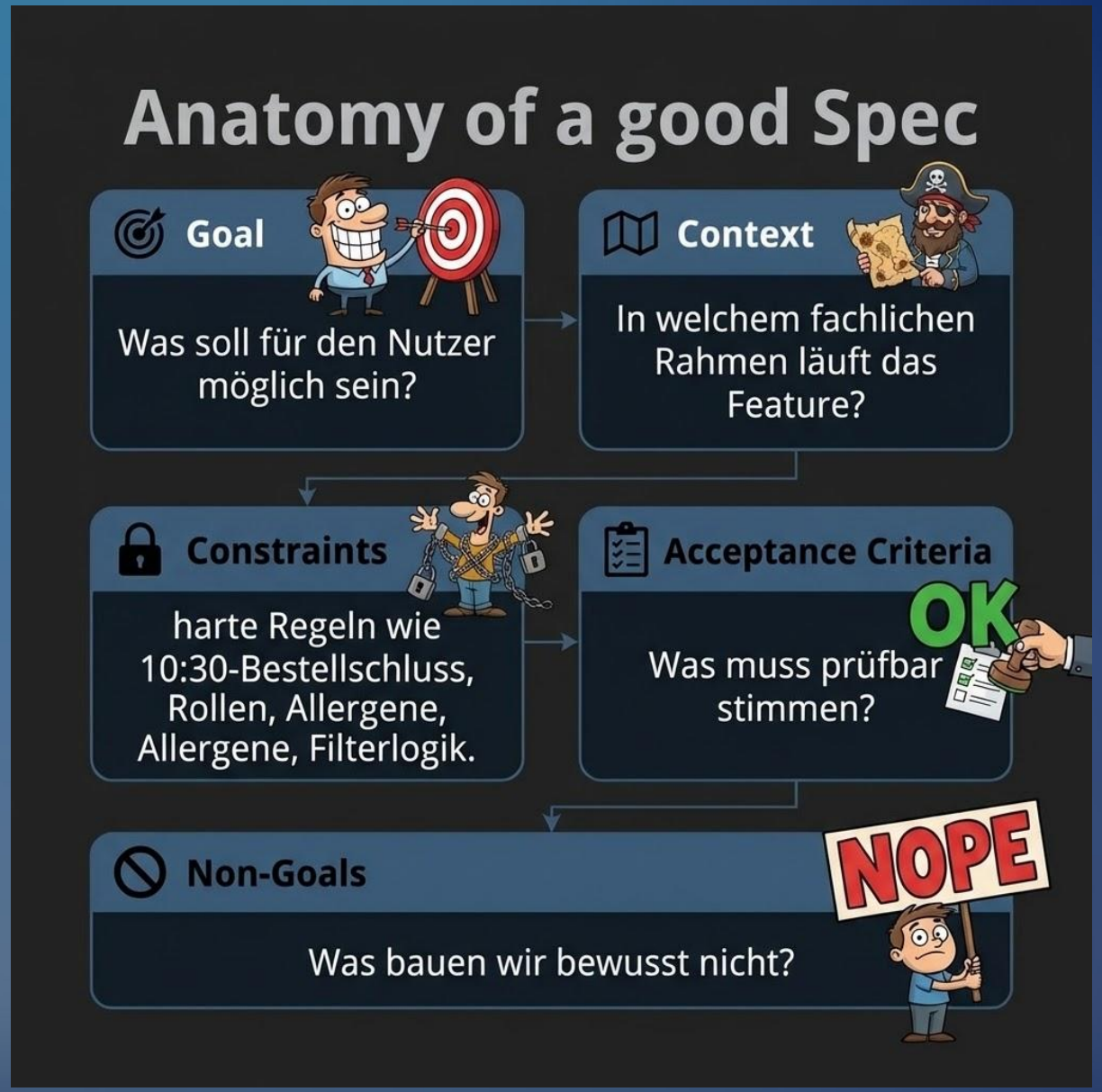
Die Greenfield-Erfahrung zeigte verschiedene Interpretationen des Tasks.

Spezifikationen müssen Ziel, Constraints und Erfolgskriterien explizit machen.

In Cursor gilt: Erst Plan, dann Code

- Für signifikante Aufgaben: Shift+Tab → Plan Mode. ↵.
- Cursor recherchiert relevante Dateien, stellt Rückfragen und erstellt einen umsetzbaren Plan mit Dateipfaden.
- Wenn der Output am Ziel vorbeigeht: nicht im Chat flicken, sondern Plan verbessern und neu starten.

Retail × AI


Anatomy of a good Spec
Goal
Was soll für den Nutzer möglich sein?
Context
In welchem fachlichen Rahmen läuft das Feature?
Constraints
harte Regeln wie 10:30-Bestellschluss, Rollen, Allergene, Allergene, Filterlogik.
Acceptance Criteria
OK
Was muss prüfbar stimmen?
Non-Goals
NOPE
Was bauen wir bewusst nicht?

Retail × AI

KONTEXT STATT KONTEXTMÜLL

⚠ CONTEXT ROT VERMEIDEN

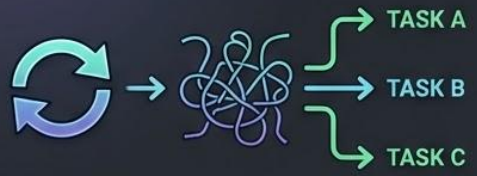
1 SMARTE EINGABE

Nicht alles in den Prompt kippen; Cursor kann Dateien oft selbst finden.

2 PRÄZISES TAGGING

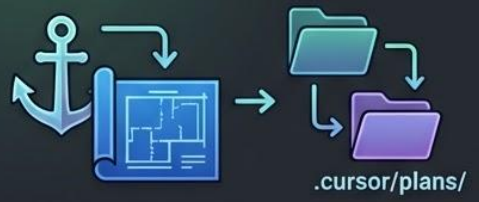
Tagge nur exakte Pfade, sensible Bereiche oder Dateien, die der Agent wiederholt verfehlt.

3 FRISCHER START



Neuer Chat bei Drift, Task-Wechsel oder sinkender Qualität.

4 STRUKTURIERTE PLANUNG



Pläne in **.cursor/plans/** speichern, damit Arbeit wiederaufnehmbar bleibt.

Spec-Driven endet nicht bei Code

- Agents arbeiten am besten gegen verifizierbare Ziele wie Tests, Typchecks und Linter.
- Cursor empfiehlt TDD-artiges Vorgehen: Tests schreiben, Fail bestätigen, dann Implementierung erzeugen.
- Review in drei Schichten: Live-Diff, Agent Review, PR-/Bugbot-Review.
- Ergebnis reviewen, nicht nur Output bewundern.

Retail × AI

shift + tab

Cursor in action

Retail × AI

4

Cursor Rules (.mdc) Betriebshandbuch

Deep Dive · 14:00–15:00
Live Exercise: markant-global.mdc

Warum Rules? Vom Chat-Gedächtnis zum Team-Standard

- Chats sind flüchtig, Rules sind dauerhaft.
- Was heute nur in deinem Kopf ist, sollte morgen in .cursor/rules/ stehen.
- Rules machen gutes Agent-Verhalten reproduzierbar über Personen, Sessions und Modelle hinweg.

Was in Rules gehört

- Commands: Build, Test, Lint, Typecheck, PR-Workflow.
- Code Patterns: Verweis auf kanonische Dateien oder bestehende Muster, nicht Copy-Paste ganzer Beispiele.
- Architektur-Konventionen: Wo Routen liegen, welche Module zuständig sind, welche Schichten benutzt werden.
- Workflow-Regeln: Zum Beispiel „nach Änderungen immer Tests/Typecheck ausführen“.

Retail × AI

Deep Dive

Cursor Rules (.mdc): Das KI-Betriebshandbuch für Markant

.cursor/rules/markant-global.mdc

```
# Commands
- docker compose up -d
- mvn test -pl edi-validator
- glab mr create --title '...'

# Code Style
- Java Error Handling: markant-commons
- Alle API-Calls: audit-geloggt (NIS-2)

# Anti-Patterns (NIEMALS)
- Kein terraform apply ohne Review
- Kein direktes DB-Write ohne Service-Layer

# Verweis auf kanonische Beispiele
- Siehe InvoiceProcessor.java
```

❌ Was NICHT in Rules kommt

- Komplette Style Guides (→ Linter)
- Jeder mögliche Befehl
- Edge Cases die selten vorkommen

🛠️ Live Exercise

- Teams bauen markant-global.mdc
- Commands + Konventionen + NIS-2-Regeln
- Anti-Pattern-Liste gemeinsam definieren
- Bestes Ruleset → ins GitLab committen

💡 *Faustregel: Rules erst hinzufügen wenn der Agent denselben Fehler wiederholt — nicht vorher über-optimieren*

So sieht ein gutes Betriebshandbuch aus

 # Commands
z. B. `docker compose up -d`, `mvn test -pl edi-validator`, `glab mr create ...`

 # Code Style / Patterns
z. B. `markant-commons` für Java Error Handling, Audit-Logging für API-Calls.

 # Workflow
wann Tests, wann Review, wann MR.

 # Anti-Patterns
z. B. kein `terraform apply` ohne Review, keine direkten DB-Writes ohne Service-Layer.

 # Canonical Examples
Verweise auf gute Referenzdateien wie `InvoiceProcessor.java`.

Wie Rules im Team entstehen
1. Sei freundlich
1. Sei freundlich
2. Keine unklaren Ausdrücke
Start einfach und minimal.
Immer dann ergänzen, wenn der Agent denselben Fehler wiederholt macht.
1. Sei freundlich
2. Keine unklaren Ausdrücke
3. Nutze Standard-Patterns
Rules ins Git committen, damit alle dieselben Standards nutzen.
Bestes Ruleset gemeinsam kuratieren statt private Prompt-Geheimnisse zu pflegen.

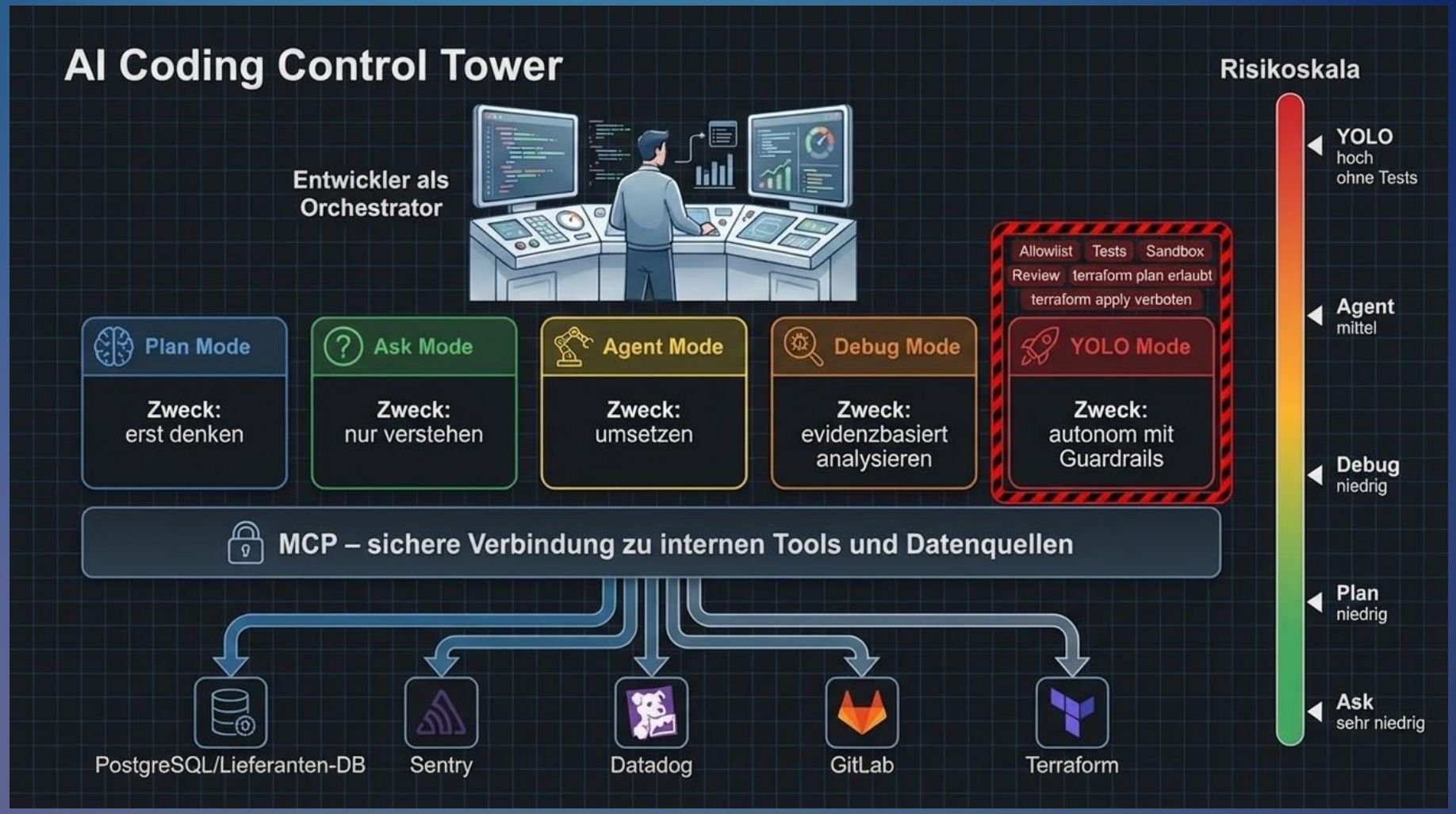
Retail × AI

5

Agent Modes, MCP & YOLO

Praxis · 15:00–15:45
MCP + Terraform Allowlist

Retail × AI


AI Coding Control Tower
Entwickler als Orchestrator
Risikoskala
YOLO
hoch
ohne Tests
Agent
mittel
Debug
niedrig
Plan
niedrig
Ask
sehr niedrig
Allowlist
Tests
Sandbox
Review
terraform plan erlaubt
terraform apply verboten
Plan Mode
Zweck:
erst denken
Ask Mode
Zweck:
nur verstehen
Agent Mode
Zweck:
umsetzen
Debug Mode
Zweck:
evidenzbasiert analysieren
YOLO Mode
Zweck:
autonom mit Guardrails
MCP – sichere Verbindung zu internen Tools und Datenquellen
PostgreSQL/Lieferanten-DB
Sentry
Datadog
GitLab
Terraform

Cursor AI-Workflow: Bausteine & Konzepte im Überblick

Plugins

Wofür gedacht?
Installierbare Pakete, die mehrere Cursor-Bausteine bündeln können (Skills, Subagents, Rules, Hooks, Commands, MCP-Server).

Wann benutzen?
Wenn du schnell eine komplette Fähigkeit oder Integration fertig einbinden willst, statt alles einzeln zu konfigurieren.

Kurzbeispiel
„Ich installiere ein Supabase- oder Sanity-Plugin und bekomme passende Bausteine als Paket."

Skills

Wofür gedacht?
Wiederverwendbare Fähigkeits- oder Wissensbausteine für bestimmte Aufgaben oder Domänen.

Wann benutzen?
Wenn der Agent für eine bestimmte Aufgabe ein wiederkehrendes Vorgehen oder spezialisiertes Wissen haben soll.

Kurzbeispiel
„Erzeuge API-Routen immer nach unserem internen Pattern."

Subagents

Wofür gedacht?
Spezialisierte Teil-Agenten für klar abgegrenzte Aufgaben.

Wann benutzen?
Wenn du unterschiedliche Agentenrollen trennen willst (z.B. UI-Agent, Refactoring-Agent, Doku-Agent).

Kurzbeispiel
„Ein Agent kümmert sich nur um Frontend, ein anderer nur um Tests."

Rules

Wofür gedacht?
Leitplanken, Konventionen und Projektregeln für das Verhalten des Modells.

Wann benutzen?
Wenn Cursor dauerhaft wissen soll, wie dein Projekt strukturiert ist, welche Patterns gelten und was vermieden werden soll.

Kurzbeispiel
„Nutze immer TypeScript strict mode, Named Exports und Zod nur an API-Grenzen."

AI Core

Hooks

Wofür gedacht?
Ereignisbasierte Skripte, die vor oder nach bestimmten Agent-Aktionen laufen.

Wann benutzen?
Wenn du Kontrolle, Prüfung oder Automation rund um den Agenten brauchst (z.B. Sicherheit, Logging, Post-Edit-Checks).

Kurzbeispiel
„Vor Shell-Ausführung prüfen, ob ein gefährlicher Befehl blockiert werden soll."

Commands

Wofür gedacht?
Direkt auslösbare Befehle oder Aktionen, die du bewusst startest.

Wann benutzen?
Wenn du etwas gezielt manuell triggern willst, statt auf automatische Agent-Entscheidungen zu warten.

Kurzbeispiel
„/create-prd" oder „/review-this-file"

MCP-Server

Wofür gedacht?
Verbindungen zu externen Tools und Diensten über das Model Context Protocol.

Wann benutzen?
Wenn Cursor auf Daten oder Aktionen außerhalb des Codes zugreifen soll (z.B. Tickets, Browser, Datenbanken, Dokus).

Kurzbeispiel
„Hole die neuesten Linear-Issues" oder „lies Inhalte aus Box/Sanity"

Retail × AI

für

Markant Gruppe

Tag 2

KI-Coding Masterclass

Vom Vibe Coding zum systematischen Context Engineering

Tag 1: Foundations & Cursor Workflows 10:00–16:00

Tag 2: Enterprise, Brownfield & CI/CD 09:00–15:00

Retail × AI

Agenda Tag 2: Enterprise Scale, Brownfield & Code Quality

09:00–09:15
Check-In & Recap Tag 1

09:15–10:30
Skalierung auf Enterprise-Codebases
Turbopuffer, AST-Chunking, .cursorignore

10:30–10:45
Kaffeepause

10:45–12:00
CI/CD-Integration & KI-gestütztes Testing
GitLab /mr · Background Agents · Bugbot

12:00–13:00
Mittagessen

13:00–14:15
Brownfield-Praxis: Legacy Code Refactoring
Plan→Test→Refactor auf echtem EDI-Code

14:15–14:30
Kaffeepause

14:30–15:00
Wrap-Up & Next Steps
5-Levels Re-Assessment · Cheat Sheet · Abschluss

09:00–15:00

Retail × AI

Checkin

Was habe ich mitgenommen?

github.com/grait-io/retail-ai-markant

Vom Vibe Coding zum Context Engineer: Dein Cursor Cheat Sheet

Die Transformation von Level 2 (Junior-Dev-Partner) zu Level 3+ (Developer als Manager) durch systematisches Kontext-Engineering.

Agent-Fehler sind keine Modell-Fehler — sie sind Kontext-Fehler.

Level 2: Der Junior-Dev-Partner (Status Quo)

Man nutzt die Ki ais Pair-Programmer im Flow State, verändert aber den grundlegenden Workflow nicht – 90% der Entwiekler stecken hier fest.

Level 3: Der Developer als Manager (Das Ziel)

Du schreibst keinen Code mehr selbst, sondern reviewst Diffs und steuerst mehrere Agent-Tabs parallel über präzise Spezifikationen.

Die Wahrnehmungsfalle

Laut METR-Studie fühlten sich Entwickler mit KI 24% schneller, waren aber tatsächlich 19% langsamer – systematischer Kontext ist die Lösung.

Workshop-Ziele vs. Cursor-Fähigkeiten Mapping

Workshop-Lernziel	Cursor-Funktion / Feature
Halluzinationen & Falschnahmen stoppen	**Cursor Rules (.cursor/rules/*.mdc):** Festlegen von Constraints und Projekt-Standards.
Proprietären Code (z.B. SPL) verstehen	**Codebase Indexing & @Docs:** Systematischer Aufbau von Referenz-Markdown-Datelen.
Kontext-Verlust (Context Rot) vermeiden	**Agent Mode & Neue Chats:** Isolierte Context Windows für spezifische Teilaufgaben nutzen.
Komplexe Architektur-Änderungen	**Spec-Driven Development:** Markdown-Specs im Plan Mode erstellen und vom Agent umsetzen lassen.
Brücke zu internen Datenquellen	**MCP (Model Contest Protocol):** Datenbanken, Wikis oder APIs direkt an den Agent anbinden.
Qualitätssicherung & Tests	**YOLO Mode + Test-Rules:** Iterative Test-Zyklen durch Agenten ohne manuelle Freigabe.

Der neue Workflow: Spec-Driven Development

Phase 1: Requirements & Design (Plan Mode)

Nutze Shift+Tab, um eine Markdown-Spezifikation zu entwerfen, bevor die erste Zeile Code generiert wird.

Phase 2: Implementation (Agent Mode)

Gib die fertige Spec als Kontext an den Agent Mode, um autonome Multi-File-Änderungen durchzuführen.

Phase 3: Validation (Debug Mode)

Nutze Agents mit Zugriff auf Runtime-Logs, um die Umsetzung direkt gegen die Spezifikation zu prüfen.

Quick-Ref: Welcher Modus für welche Aufgabe?

Ask Mode (Strg+L)

Ideal für „Read-only" Untersuchungen und das Verständnis der Codebase ohne Änderungen.

Agent Mode (Strg+I)

Der Standard für autonome Aufgaben, die Änderungen an mehreren Dateien erfordern.

Plan Mode

Beste Wahl für die strategische Planung komplexer Features vor der Umsetzung.

YOLO Mode

Aktiviert autonome Terminal-Befehle für schnelle TDD-Zyklen (Test Driven Development).

NotebookLM

Retail × AI

6

Enterprise Codebases

Tag 2 · 09:15–10:30
Turbopuffer · AST-Chunking

Retail × AI

Tag 2 · 09:15

Skalierung auf Enterprise-Codebases

Warum das bei Markant kritisch ist

- 15.000+ Lieferanten-Integrationen = massive Codebase
- Multi-Repo Setup: EDI, PIM, Compliance getrennt
- Jahrzehnte alter EDI-Code kaum durchsuchbar
- Neue Devs brauchen Wochen zum Onboarding

Cursor-Lösungen

- Turbopuffer: 100B+ Vektoren, Sub-ms-Suche
- AST-Chunking: Code semantisch zerlegt, nicht nur Text
- .cursorignore: node_modules, Vendor-Code ausschließen
- Codebase Indexing: einmal konfigurieren, alle profitieren
- "Wie funktioniert INVOIC-D96A?" → Agent antwortet in Sek.
- MCP - Kern-Bibliotheken & ggf. Markant MCP-Server

💡 *Live: Großes Open-Source-Repo indexieren, .cursorignore zeigen, Suchqualität demonstrieren*

Retail × AI

Tag 2 · 10:45

7

CI/CD & KI-Testing

Tag 2 · 10:45–12:00
GitLab /mr · Bugbot

Tag 2 · 10:45

CI/CD-Integration & KI-gestütztes Testing

⚠️ **Markant nutzt GitLab — alle Demos laufen mit glab CLI + GitLab CI, nicht GitHub Actions**

`/mr`	Commit → Push → GitLab MR erstellen + Issue-Referenz
`/write-tests`	Bestehende Kantinen-Planer Endpunkte → Tests generieren
`/fix-issue [n]`	GitLab Issue holen → Code finden → Fix → MR öffnen
`/security-scan`	NIS-2 Compliance-Check auf geänderten Files vor MR

💡 *Bugbot auf jeden GitLab MR konfigurieren — automatische Code-Reviews ohne Mehraufwand*

Retail × AI

8

Tag 2 · 13:00

Brownfield-Praxis: Legacy Refactoring

Tag 2 · 13:00–14:15
Plan → Test → Refactor

Retail × AI

Tag 2 · 13:00

Brownfield-Praxis: Legacy EDI Refactoring

1 PLAN

Shift+Tab → Plan Mode
Agent analysiert Legacy-EDI
Refactoring-Plan mit Dateipfaden
In .cursor/plans/ speichern

2 TEST

Agent schreibt Tests für
bestehendes EDIFACT-Verhalten
Erst dann Code anfassen
Tests committen

3 REFACTOR

Agent refaktoriert gegen Tests
Grüne Tests = Erfolgskriterium
Kein YOLO ohne Tests
GitLab MR + Bugbot-Review

💡 *Legacy-Codebase vorbereiten: ~500 Zeilen, 5 bewusste Probleme — jede Gruppe tackled eines mit dem Dreischritt*

Retail × AI

Wrap-Up & Next Steps

Rules ins Git

markant-rules.mdc heute noch committen

YOLO Allowlist

terraform plan ja, terraform apply nein — Konsens herstellen

Plan First

Shift+Tab vor jeder EDI-Migration oder NIS-2-Änderung

Context Rot

1 Chat = 1 Task. Bei Drift: neuen Chat starten

GitLab /mr

.cursor/commands/mr.md + /security-scan einrichten

30-Tage-Plan

Welche 3 Use Cases setzt ihr in 30 Tagen um?

EDI Spec-Driven

Nächsten EDIFACT-Validator als Markdown-Spec schreiben

→ Re-Assessment: 5 Levels
Markant Umfrage umfrage beantworten

Retail × AI

Dein Feedback zur KI-Coding Masterclass